JavaScript Praxisaufgaben

Arrays, Objekte, map / filter / reduce und DOM

1

Aufgabe 1: Die Raumstation Elysium

Notfallmanagement | Mittel

2

Aufgabe 2: Dungeon Manager

Das Inventar eines Abenteurers | Mittel bis anspruchsvoll

AUFGABE 1

Die Raumstation Elysium

Schwierigkeitsgrad: Mittel

Szenario

MISSION

Du bist als Junior-Entwickler für die Bordsoftware einer Raumstation verantwortlich. Nach einem technischen Zwischenfall sind mehrere Systeme beschädigt. Die Crew benötigt eine Übersicht, um zu entscheiden, welche Probleme zuerst behoben werden müssen.

Ausgangsdaten

Lege ein Array mit **mindestens sechs Systemen** an:

```
const systeme = [
    {
        name: "Sauerstoffversorgung",
        status: "kritisch",
        energieverbrauch: 80,
        reparaturkosten: 1200
    },
    {
        name: "Kantine",
        status: "normal",
        energieverbrauch: 30,
        reparaturkosten: 100
    },
    {
        name: "Antrieb",
        status: "kritisch",
        energieverbrauch: 95,
        reparaturkosten: 5000
    },
    {
        name: "Kommunikation",
        status: "gestört",
        energieverbrauch: 40,
        reparaturkosten: 800
    }
];
```

Arbeitsaufträge

1. **Systemübersicht**
   Gib alle Systeme in einer HTML-Liste aus. Jedes System soll seinen Namen und Status anzeigen.

2. **Notfallfilter**
   Zeige nur Systeme mit dem Status `kritisch` an. Nutze dafür `filter()`.

3. **Reparaturbudget**
   Berechne mit `reduce()` die gesamten Reparaturkosten aller Systeme.

4. **Energiebericht**
   Erstelle mit `map()` eine neue Liste, die für jedes System den Namen und den Energieverbrauch enthält.

5. **Notfallmeldung**
   Markiere kritische Systeme im DOM **rot** und gestörte Systeme **orange**.

6. **Reparatur durchführen**
   Füge jedem System einen Button hinzu. Ein Klick ändert den Status des Systems auf `normal` und aktualisiert die Anzeige.

7. **Statistik**
   Zeige an, wie viele Systeme kritisch, gestört und normal sind.

Technische Anforderungen

- [ ] Verwende Objekte und Arrays.
- [ ] Nutze mindestens einmal `map()`, `filter()` und `reduce()`.
- [ ] Verwende Event-Listener statt Inline-Event-Handler.
- [ ] Die HTML-Ausgabe muss nach jeder Reparatur aktualisiert werden.

BONUS

Die Reparatur eines Systems kostet Geld. Ist das Budget aufgebraucht, dürfen keine weiteren Reparaturen durchgeführt werden.

AUFGABE 2

Dungeon Manager

Schwierigkeitsgrad: Mittel bis anspruchsvoll

Das Inventar eines Abenteurers

Szenario

QUEST

Du entwickelst ein Inventarsystem für ein Rollenspiel. Der Spieler erkundet einen Dungeon, sammelt Gegenstände und muss entscheiden, was er behält, verkauft oder verwendet.

Ausgangsdaten

```
const inventar = [
    {
        id: 1,
        name: "Eisenschwert",
        typ: "waffe",
        wert: 150,
        gewicht: 4,
        seltenheit: "gewöhnlich"
    },
    {
        id: 2,
        name: "Heiltrank",
        typ: "trank",
        wert: 50,
        gewicht: 1,
        seltenheit: "gewöhnlich"
    },
    {
        id: 3,
        name: "Drachenschild",
        typ: "rüstung",
        wert: 900,
        gewicht: 8,
        seltenheit: "episch"
    },
    {
        id: 4,
        name: "Magischer Ring",
        typ: "schmuck",
        wert: 1200,
        gewicht: 0.2,
        seltenheit: "legendär"
    }
];
```

Arbeitsaufträge

1. **Inventar anzeigen**
   Stelle alle Gegenstände als Karten mit Name, Typ, Wert, Gewicht und Seltenheit dar.

2. **Kategorien**
   Ergänze Filterbuttons für Waffen, Tränke, Rüstung und Schmuck.

3. **Sortierung**
   Ermögliche die Sortierung nach Wert und Gewicht. Verwende `sort()`.

4. **Vermögen berechnen**
   Berechne den Gesamtwert des Inventars mit `reduce()`.

5. **Überladung verhindern**
   Der Spieler darf maximal **20 Gewichtseinheiten** tragen. Zeige das aktuelle Gewicht und eine Warnung bei Überschreitung an.

6. **Gegenstände verkaufen**
   Jeder Gegenstand erhält einen Verkaufsbutton. Beim Verkauf wird der Gegenstand aus dem Inventar entfernt und sein Wert dem Goldbestand hinzugefügt.

7. **Tränke verwenden**
   Wird ein Heiltrank verwendet, verschwindet er aus dem Inventar. Andere Gegenstände dürfen nicht auf diese Weise verwendet werden.

8. **Inventarstatus**
   Zeige an, wie viele Gegenstände sich im Inventar befinden und wie viel Gold der Spieler besitzt.

Technische Anforderungen

- [ ] Die Gegenstände müssen über ihre `id` eindeutig identifizierbar sein.
- [ ] Verwende `filter()` zum Filtern und Entfernen von Gegenständen.
- [ ] Verändere nicht nur die HTML-Darstellung, sondern auch die zugrunde liegenden Daten.
- [ ] Die Anzeige muss nach jeder Aktion konsistent bleiben.

> **BONUS**
>
> Füge ein Suchfeld hinzu, mit dem Gegenstände anhand ihres Namens gefunden werden können.